Unit 5

Connecting with Database

- Python being a high-level language provides support for various databases.
- We can connect and run queries for a particular database using Python and without writing raw queries in the terminal or shell of that particular database, we just need to have that database installed in our system.

- A database is an organized collection of data stored electronically. It is managed by a Database Management System (DBMS) that allows users to access, update, and manage data efficiently. Databases are primarily categorized into two types:

1. Relational Databases (RDBMS)

2. Non-relational Databases (NoSQL)

1. Relational Databases (RDBMS)

- Store data in structured tables with rows and columns. They support relationships between data entities. Examples:
- MySQL
- PostgreSQL
- SQLite

2. Non-relational Databases (NoSQL)

- Store data in various formats like documents, key-value pairs, or graphs, offering more flexibility. For Examples:
- MongoDB
- Cassandra

Installation Instructions

- Install MySQL Connector for Python
- Once Python and PIP are installed, we can proceed with installing the MySQL Connector package for Python.

- For Windows
- mysql-connector method can be installed on Windows with the use of following command:
- ***pip install mysql-connector-python***

Connecting to the Database

- The **mysql.connector** library provides the **connect()** method, which is used to establish a connection between the MySQL database and a Python application. This allows you to connect to both **local** and **cloud-based** MySQL databases effortlessly.
- ***Conn_obj = mysql.connector.connect(***
  host=<hostname>,
  user=<username>,
  passwd=<password>,
  database=<database_name> # optional

)

- **connect(**) function accepts the following arguments.
- **Hostname:** It represents the server name or IP address on which MySQL is running.
- **Username:** It represents the name of the user that we use to work with the MySQL server. By default, the username for the MySQL database is root.
- **Password:** The password is provided at the time of installing the MySQL database. We don't need to pass a password if we are using the root.
- **Database:** It specifies the database name which we want to connect. This argument is used when we have multiple databases.

Create Database

- In Python, a MySQL database can be created using the CREATE DATABASE statement. The mysql.connector module is used to connect Python programs with the MySQL server and execute database creation queries.
- Syntax
- CREATE DATABASE database_name

```
import mysql.connector

dataBase = mysql.connector.connect(
    host="localhost",
    user="root",
    passwd="1234"
)

cursorObject = dataBase.cursor()

cursorObject.execute("CREATE DATABASE company")
print("Database created successfully.")
```

Explanation:

- mysql.connector.connect() connects Python with the MySQL server.
- cursor() creates a cursor object used to execute SQL queries.
- CREATE DATABASE company creates a database named company.
- print() displays the success message after database creation.

Create Table

- MySQL is a Relational Database Management System (RDBMS) whereas the structured Query Language (SQL) is the language used for handling the RDBMS using commands
- i.e Creating, Inserting, Updating and Deleting (CRUD) the data from the databases. SQL commands are case insensitive i.e "CREATE" and "create" signify the same command.

- The table is a collection of data organized in the form of rows and columns.
- Rows are also called tuples
- Columns are called the attributes of the table
- **Syntax:**
- As we have discussed that tables are created inside a database so we need to first make sure to select or move into to the database before creating a table. To select the "College" database use this command:
- ***USE college;***

- This will select the "College" database, now we can proceed to create tables inside it. Here's how to create a table:
- *CREATE TABLE table_name*
  (
  column_name_1 column_Data_type,
  column_name_2 column_Data_type,
  :
  :
  column_name_n column_Data_type
  );

- **SQL Data Types:**
- **Numeric:** INT, FLOAT, DOUBLE
- **Character/String:** VARCHAR, CHAR, TEXT
- **Date/Time:** DATE, TIME, DATETIME
- **Unicode:** NCHAR, NVARCHAR
- **Binary:** BLOB

Insert into Table

- INSERT INTO statement is used to add new records to a MySQL table. In Python, you can execute INSERT queries using the mysql.connector module.
- After inserting data, you must call the commit() method to permanently save the changes to the database.
- Syntax
- The general syntax of the SQL INSERT INTO statement is:

```
INSERT INTO table_name (column1, column2, ...) VALUES (value1, value2, ...);
```

- For parameterized queries in Python:

- query = "INSERT INTO Student (Name, Course, Marks) VALUES (%s, %s, %s)"
- values = ("Emma", "CSE", 92)

- cursor.execute(query, values)
- connection.commit()

- **Insert Multiple Records**
- The example below inserts multiple student records using the executemany() method.

```
cursor = connection.cursor()
query = """
INSERT INTO Student (Name, Course, Marks)
VALUES (%s, %s, %s)
"""
students = [
    ("Charlotte", "IT", 87),
    ("Benjamin", "ECE", 81),
    ("Amelia", "CSE", 94)
]

cursor.executemany(query, students)
connection.commit()
print(cursor.rowcount, "records inserted.")
```

Select Query

- SELECT statement is used to retrieve data from one or more tables in a MySQL database. In Python, you can execute SELECT queries using the mysql.connector module to fetch records from the database and process them within your application.
- Syntax
- The general syntax of the SQL SELECT statement is:

- SELECT column_name(s)
- FROM table_name;

- To retrieve all columns from a table, use the * wildcard.

- SELECT * FROM table_name;

```
cursor = conn.cursor()
query = "SELECT * FROM Student"
cursor.execute(query)

for row in cursor.fetchall():
    print(row)
```

- **Select Specific Columns**
- The example below retrieves only the Name and Course columns from the table.

```
cursor = conn.cursor()
query = "SELECT Name, Course FROM Student"
cursor.execute(query)

for row in cursor.fetchall():
    print(row)
```

- **Select Records with Sorting**
- The example below retrieves all students and sorts them in descending order of marks.

```
cursor = conn.cursor()
query = "SELECT Name, Marks FROM Student ORDER BY Marks DESC"
cursor.execute(query)

for row in cursor.fetchall():
    print(row)
```

Update Query

- The UPDATE query in SQL is used to modify existing records in a table. It allows you to update specific columns' values in one or more rows of a table. It's important to note that the **UPDATE** query **affects only the data**, **not the structure** of the table.
- Syntax
- UPDATE tablename SET column1 = "new_value", column2 = "new_value“ WHERE condition;
- **tablename:** The name of the table where the data will be updated.
- **SET:** Specifies the columns and their new values.
- **WHERE:** Determines which rows will be updated. If omitted, all rows in the table will be updated.

- # Create a cursor object
- cs = mydb.cursor()

- # SQL query to update the age of a student
- statement = "UPDATE STUDENT SET AGE = 23 WHERE Name = 'Rishi Kumar'"

- # Execute the update statement
- cs.execute(statement)

- # Commit the changes to the database
- mydb.commit()

Delete Query

- In MySQL, the **DELETE** query is used to remove rows from a table based on a specific condition defined in the **WHERE** clause. Without a **WHERE** clause, the entire table will be emptied.
- Syntax
- DELETE FROM table_name WHERE condition;
- **table_name:** The name of the table from which records are to be deleted.
- **condition:** The condition that specifies which rows should be deleted.

- delete_query = "DELETE FROM STUDENT WHERE ROLL = %s"
- delete_value = (1,)
- cursorObject.execute(delete_query, delete_value)

Drop Table

- A connector is employed when we have to use MySQL with other programming languages. The work of MySQL-connector is to provide access to MySQL Driver to the required language. Thus, it generates a connection between the programming language and the MySQL Server.
- **Drop Table Command**
- Drop command affects the structure of the table and not data. It is used to delete an already existing table. For cases where you are not sure if the table to be dropped exists or not **DROP TABLE IF EXISTS** command is used.

```
cs = mydb.cursor()

# Drop table if it exists
statement = "DROP TABLE IF EXISTS Employee"

# Executing the query
cs.execute(statement)
mydb.commit()
```

Commit & RollBack Operation in Python

- In Python, the **commit()** and **rollback()** methods play an essential role in database transactions, ensuring atomicity, consistency, isolation, and durability (**ACID properties**). These methods help manage changes to the database in a reliable way, ensuring data integrity even in case of errors or failures.

1. commit() method

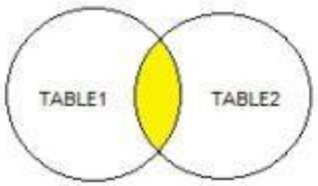
- The commit() method is used to save changes made to the database permanently. It confirms that all the changes executed during the current session, such as INSERT, UPDATE, or DELETE queries, should be saved in the database. Without commit(), any changes would be lost once the connection is closed.
- *comm.commit()*
- **comm:** refers to the database connection object
- **commit()** method ensures that changes are persistent in the database.
- **commit()** method ensures that this update is saved permanently to the College database.

2. rollback() method

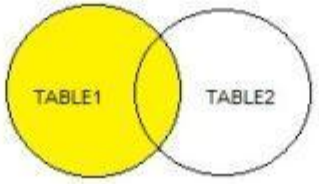
- The rollback() method is used to undo changes made during the current transaction. If a condition arises where one is not satisfied with the changes made to the database or a database transaction fails, the rollback() method can be used to retrieve the original data that was changed through the commit() method.
- Syntax
- comm.rollback()
- **comm:** refers to the database connection object
- **rollback()** method discards all changes since the last commit.

Python MySQL - Join

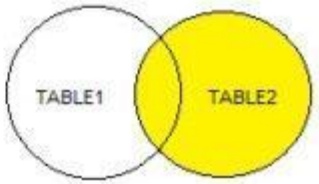
- **JOIN** clause is used to combine rows from two or more tables based on a related column. It allows you to query data from multiple tables and return the result as a single set. There are several types of joins in SQL, each providing different results based on how data is combined.



INNER JOIN



LEFT JOIN



RIGHT JOIN

1. INNER JOIN

- Returns records that have matching values in both tables. If no match is found, no record is returned.
- Syntax:
- SELECT column1, column2... FROM table1 JOIN table2 ON table1.column = table2.column;

```
cs = mydb.cursor()

# INNER JOIN: only students with matching details
statement = """SELECT s.Name, d.Branch
            FROM Student s
            INNER JOIN StudentDetails d ON s.Roll_no = d.Roll_no"""

cs.execute(statement)
```

- 2. LEFT JOIN
- Returns all records from the left table and the matched records from the right table. If no match is found, the result is NULL on the right side.
- Syntax:
- SELECT column1, column2... FROM table1 LEFT JOIN table2 ON table1.column = table2.column;
- cs = mydb.cursor()

- # Left join
- statement = """SELECT s.Name, d.Section
- FROM Student s
- LEFT JOIN StudentDetails d ON s.Roll_no = d.Roll_no"""

- cs.execute(statement)

- 3. RIGHT JOIN
- Returns all records from the right table and the matched records from the left table. If no match is found, the result is NULL on the left side.
- Syntax:
- SELECT column1, column2... FROM table1 RIGHT JOIN table2 ON table1.column = table2.column;
- cs = mydb.cursor()

- #Right Join
- statement = """SELECT d.Branch, s.Name
- FROM Student s
- RIGHT JOIN StudentDetails d ON s.Roll_no = d.Roll_no"""

- cs.execute(statement)

EX.1 CRUD WITH UI

```
import tkinter as tk
from tkinter import ttk, messagebox
import mysql.connector


# ---------------- MYSQL CONNECTION ----------------

conn = mysql.connector.connect(
    host="localhost",
    user="root",
    password="",
    database="studentdb"
)

cursor = conn.cursor()


# ---------------- TKINTER WINDOW ----------------

root = tk.Tk()

root.title("Student CRUD Application")

root.geometry("900x550")


selected_id = None


# ---------------- FUNCTIONS ----------------

def clear_fields():

    firstname_entry.delete(0, tk.END)
    lastname_entry.delete(0, tk.END)
    mobile_entry.delete(0, tk.END)
    email_entry.delete(0, tk.END)


def display_data():

    for item in tree.get_children():
        tree.delete(item)

    cursor.execute("SELECT * FROM students")

    rows = cursor.fetchall()

    for row in rows:
        tree.insert(
            "",
            tk.END,
            values=row
        )


def insert_data():

    firstname = firstname_entry.get()
```

```
    lastname = lastname_entry.get()
    mobile = mobile_entry.get()
    email = email_entry.get()

    if firstname == "" or lastname == "":

        messagebox.showerror(
            "Error",
            "Please enter First Name and Last Name"
        )

        return

    query = """
        INSERT INTO students
        (firstname, lastname, mobile, email)

        VALUES (%s, %s, %s, %s)
    """

    values = (
        firstname,
        lastname,
        mobile,
        email
    )

    cursor.execute(
        query,
        values
    )

    conn.commit()

    messagebox.showinfo(
        "Success",
        "Student Added Successfully"
    )

    clear_fields()

    display_data()


def select_data(event):

    global selected_id

    selected = tree.focus()

    if selected:

        values = tree.item(
            selected,
            "values"
        )

        selected_id = values[0]

        clear_fields()
```

```
        firstname_entry.insert(
            0,
            values[1]
        )

        lastname_entry.insert(
            0,
            values[2]
        )

        mobile_entry.insert(
            0,
            values[3]
        )

        email_entry.insert(
            0,
            values[4]
        )


def update_data():

    global selected_id

    if selected_id is None:

        messagebox.showerror(
            "Error",
            "Please select a student"
        )

        return

    query = """
        UPDATE students

        SET firstname=%s,
            lastname=%s,
            mobile=%s,
            email=%s

        WHERE id=%s
    """

    values = (
        firstname_entry.get(),
        lastname_entry.get(),
        mobile_entry.get(),
        email_entry.get(),
        selected_id
    )

    cursor.execute(
        query,
        values
    )

    conn.commit()
```

```
    messagebox.showinfo(
        "Success",
        "Student Updated Successfully"
    )

    clear_fields()

    display_data()


def delete_data():

    global selected_id

    if selected_id is None:

        messagebox.showerror(
            "Error",
            "Please select a student"
        )

        return

    confirm = messagebox.askyesno(
        "Confirm Delete",
        "Do you want to delete this student?"
    )

    if confirm:

        query = """
            DELETE FROM students
            WHERE id=%s
        """

        cursor.execute(
            query,
            (selected_id,)
        )

        conn.commit()

        messagebox.showinfo(
            "Success",
            "Student Deleted Successfully"
        )

        clear_fields()

        display_data()


# ---------------- FORM ----------------

tk.Label(
    root,
    text="First Name"
).place(
    x=50,
```

```
    y=50
)

firstname_entry = tk.Entry(
    root,
    width=30
)

firstname_entry.place(
    x=180,
    y=50
)


tk.Label(
    root,
    text="Last Name"
).place(
    x=50,
    y=100
)

lastname_entry = tk.Entry(
    root,
    width=30
)

lastname_entry.place(
    x=180,
    y=100
)


tk.Label(
    root,
    text="Mobile"
).place(
    x=50,
    y=150
)

mobile_entry = tk.Entry(
    root,
    width=30
)

mobile_entry.place(
    x=180,
    y=150
)


tk.Label(
    root,
    text="Email"
).place(
    x=50,
    y=200
)

email_entry = tk.Entry(
```

```
    root,
    width=30
)

email_entry.place(
    x=180,
    y=200
)


# ---------------- BUTTONS ----------------

tk.Button(
    root,
    text="Add",
    command=insert_data
).place(
    x=180,
    y=250
)


tk.Button(
    root,
    text="Update",
    command=update_data
).place(
    x=250,
    y=250
)


tk.Button(
    root,
    text="Delete",
    command=delete_data
).place(
    x=330,
    y=250
)


tk.Button(
    root,
    text="Clear",
    command=clear_fields
).place(
    x=410,
    y=250
)


# ---------------- TREEVIEW ----------------

columns = (
    "ID",
    "First Name",
    "Last Name",
    "Mobile",
    "Email"
)
```

```
tree = ttk.Treeview(
    root,
    columns=columns,
    show="headings"
)


for column in columns:

    tree.heading(
        column,
        text=column
    )


tree.column(
    "ID",
    width=50
)


tree.place(
    x=400,
    y=50,
    width=480,
    height=400
)


tree.bind(
    "<ButtonRelease-1>",
    select_data
)


# ---------------- LOAD DATA ----------------

display_data()


# ---------------- RUN ----------------

root.mainloop()
```

Ex. 2 CRUD WITHOUT UI WITHOUT MENU

```
import mysql.connector


# ----------------------------------
# STEP 1: CONNECT TO MYSQL SERVER
# ----------------------------------

conn = mysql.connector.connect(
    host="localhost",
    user="root",
    password=""
)

cursor = conn.cursor()

print("MySQL Connected Successfully")


# ----------------------------------
# STEP 2: CREATE DATABASE
# ----------------------------------

cursor.execute(
    "CREATE DATABASE IF NOT EXISTS studentdb"
)

print("Database Created Successfully")


# ----------------------------------
# STEP 3: USE DATABASE
# ----------------------------------

cursor.execute(
    "USE studentdb"
)


# ----------------------------------
# STEP 4: CREATE TABLE
# ----------------------------------

cursor.execute("""
CREATE TABLE IF NOT EXISTS students (

    id INT AUTO_INCREMENT PRIMARY KEY,

    firstname VARCHAR(100),
    lastname VARCHAR(100),
    mobile VARCHAR(15),
    email VARCHAR(100)
)
""")

print("Table Created Successfully")


# ----------------------------------
# STEP 5: INSERT DATA (CREATE)
```

```
# ----------------------------------

insert_query = """
INSERT INTO students
(firstname, lastname, mobile, email)

VALUES (%s, %s, %s, %s)
"""


insert_values = (
    "Yash",
    "Kotecha",
    "9876543210",
    "yash@gmail.com"
)


cursor.execute(
    insert_query,
    insert_values
)

conn.commit()

print("Data Inserted Successfully")


# ----------------------------------
# STEP 6: READ DATA
# ----------------------------------

cursor.execute(
    "SELECT * FROM students"
)

rows = cursor.fetchall()


print("\n----- STUDENT DATA -----")

for row in rows:
    print(row)


# ----------------------------------
# STEP 7: UPDATE DATA
# ----------------------------------

update_query = """ UPDATE students SET mobile = %s WHERE id = %s"""
update_values = ("9999999999",3)
cursor.execute(  update_query, update_values)

conn.commit()

print("\nData Updated Successfully")


# ----------------------------------
# STEP 8: READ UPDATED DATA
# ----------------------------------
```

```
cursor.execute(
    "SELECT * FROM students"
)

rows = cursor.fetchall()


print("\n----- UPDATED STUDENT DATA -----")

for row in rows:
    print(row)


# ----------------------------------
# STEP 9: DELETE DATA
# ----------------------------------

delete_query = """DELETE FROM students WHERE id = %s """


delete_values = (5,)


cursor.execute(
    delete_query,
    delete_values
)

conn.commit()

print("\nData Deleted Successfully")


# ----------------------------------
# STEP 10: FINAL READ
# ----------------------------------

cursor.execute(
    "SELECT * FROM students"
)

rows = cursor.fetchall()


print("\n----- FINAL STUDENT DATA -----")

for row in rows:
    print(row)


# ----------------------------------
# STEP 11: CLOSE CONNECTION
# ----------------------------------

cursor.close()

conn.close()

print("\nMySQL Connection Closed")
```

Ex. 3 CRUD WITHOUT UI WITH MENU

```
import mysql.connector


# ========================================
# 1. CONNECT TO MYSQL
# ========================================

conn = mysql.connector.connect(
    host="localhost",
    user="root",
    password=""
)

cursor = conn.cursor()

print("MySQL Connected Successfully!")


# ========================================
# 2. CREATE DATABASE
# ========================================

cursor.execute("CREATE DATABASE IF NOT EXISTS studentdb")

print("Database Checked/Created Successfully")


# ========================================
# 3. USE DATABASE
# ========================================

cursor.execute("USE studentdb")


# ========================================
# 4. CREATE TABLE
# ========================================

cursor.execute("""
CREATE TABLE IF NOT EXISTS students (
    id INT AUTO_INCREMENT PRIMARY KEY,
    firstname VARCHAR(50),
    lastname VARCHAR(50),
    mobile VARCHAR(15),
    email VARCHAR(100)
)
""")

print("Table Checked/Created Successfully")


# ========================================
# INSERT STUDENT
# ========================================

def insert_student():
```

```
    print("\n----- INSERT STUDENT -----")

    firstname = input("Enter First Name: ")
    lastname = input("Enter Last Name: ")
    mobile = input("Enter Mobile: ")
    email = input("Enter Email: ")

    query = """
    INSERT INTO students
    (firstname, lastname, mobile, email)
    VALUES (%s, %s, %s, %s)
    """

    values = (
        firstname,
        lastname,
        mobile,
        email
    )

    cursor.execute(query, values)

    conn.commit()

    print("Student inserted successfully!")

    print("Student ID:", cursor.lastrowid)


# ========================================
# DISPLAY ALL STUDENTS
# ========================================

def display_students():

    print("\n----- ALL STUDENTS -----")

    cursor.execute("SELECT * FROM students")

    rows = cursor.fetchall()

    if len(rows) == 0:

        print("No student records found.")

    else:

        for row in rows:
            print(
                "ID:", row[0],
                "| First Name:", row[1],
                "| Last Name:", row[2],
                "| Mobile:", row[3],
                "| Email:", row[4]
            )


# ========================================
# UPDATE STUDENT
```

```python
# ========================================

def update_student():

    print("\n----- UPDATE STUDENT -----")

    student_id = input("Enter Student ID: ")

    firstname = input("Enter New First Name: ")
    lastname = input("Enter New Last Name: ")
    mobile = input("Enter New Mobile: ")
    email = input("Enter New Email: ")

    query = """
    UPDATE students
    SET firstname = %s,
        lastname = %s,
        mobile = %s,
        email = %s
    WHERE id = %s
    """

    values = (
        firstname,
        lastname,
        mobile,
        email,
        student_id
    )

    cursor.execute(query, values)

    conn.commit()

    if cursor.rowcount > 0:

        print("Student updated successfully!")

    else:

        print("Student ID not found.")


# ========================================
# DELETE STUDENT
# ========================================

def delete_student():

    print("\n----- DELETE STUDENT -----")

    student_id = input("Enter Student ID: ")

    query = """
    DELETE FROM students
    WHERE id = %s
    """

    values = (student_id,)
```

```
    cursor.execute(query, values)

    conn.commit()

    if cursor.rowcount > 0:

        print("Student deleted successfully!")

    else:

        print("Student ID not found.")


# ========================================
# MENU
# ========================================

while True:

    print("\n================================")
    print("      STUDENT MANAGEMENT")
    print("================================")

    print("1. Insert Student")
    print("2. Display All Students")
    print("3. Update Student")
    print("4. Delete Student")
    print("5. Exit")

    print("================================")

    choice = input("Enter your choice: ")


    # INSERT

    if choice == "1":

        insert_student()


    # READ

    elif choice == "2":

        display_students()


    # UPDATE

    elif choice == "3":

        update_student()


    # DELETE

    elif choice == "4":

        delete_student()
```

```
    # EXIT
    elif choice == "5":
        print("Program closed.")
        break

    else:
        print("Invalid choice! Please try again.")

# ========================================
# CLOSE CONNECTION
# ========================================
cursor.close()
conn.close()
print("MySQL connection closed.")
```